



**UNIVERSIDAD
DE ANTIOQUIA**

LÓGICA Y REPRESENTACIÓN I

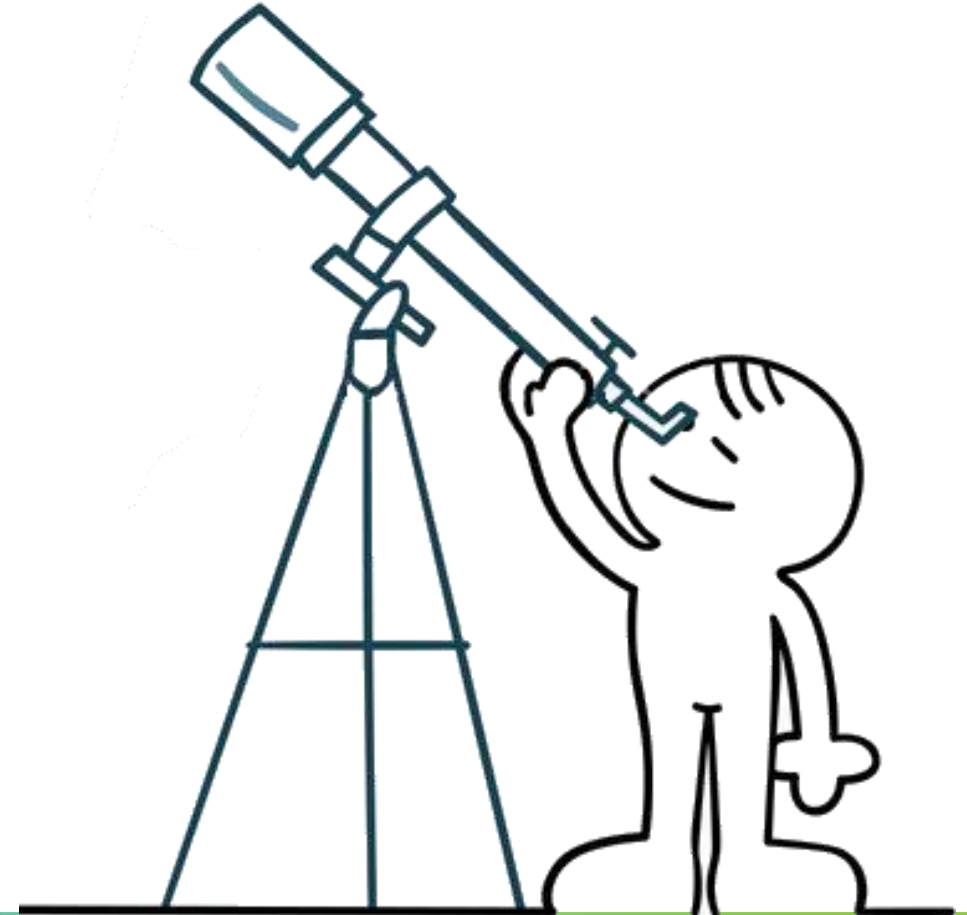
PROGRAMA DE INGENIERÍA DE SISTEMAS

Material desarrollado por **Carlos Andrés Mera Banguero**

<https://github.com/carlosmera20/>

🦋 ESTRUCTURAS DE ALMACENAMIENTO ESTÁTICAS – ARREGLOS BIDIMENSIONALES

- ✓ Declaración de arreglos bidimensionales
- ✓ Acceso a los datos en arreglos bidimensionales
- ✓ Recorrido de arreglos bidimensionales
- ✓ Promedios en Filas y Columnas



SOBRE LAS MATRICES



ARREGLOS BIDIMENSIONALES



Un **arreglo bidimensional** (o **matriz**) es un arreglo con dos dimensiones que están representadas por **filas** y **columnas**. Este tipo de arreglo se puede comparar con una matriz.

					Columna
					↓
	2.8	3.0	4.7	5.0	2.5
Fila →	3.9	4.1	3.2	4.0	3.0
	4.7	2.3	5.0	3.6	4.9

Arreglo **Bidimensional** con tres (3) Filas y cinco (5) Columnas

ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✚ Al igual que en los arreglos unidimensionales, las casillas de una matriz están definidas por el número de la fila y de la columna de estas.

	0	1	2	3	4
0	2.8	3.0	4.7	5.0	2.5
1	3.9	4.1	3.2	4.0	3.0
2	4.7	2.3	5.0	3.6	4.9

→ Esta casilla está representada en la matriz por la fila 1 y la columna 4

- ✚ Para declarar una matriz se debe especificar su nombre, el número de filas, el número de columnas y el tipo de datos de sus elementos. El tipo de datos debe ser siempre entero, real, lógico o carácter.

PSEUDOCÓDIGO

Real matriz(3, 5)
↓ ↓ ↓
Variable Filas Columnas

Python

matriz = np.full((3,5), fill_value=0, dtype=float)
↓ ↓ ↓
Variable Filas Columnas



LECTURA Y ESCRITURA DE DATOS EN MATRICES

ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para almacenar o leer un valor de una casilla de la matriz se debe especificar la fila y la columna. Por ejemplo, para obtener la siguiente matriz se puede hacer lo siguiente:

	0	1	2	3	4
0	19		8		
1					48
2			15	24	

PSEUDOCÓDIGO

```
Entero matriz(3)(5)

matriz[0,0] = 19
matriz[2,2] = 15
matriz[2,3] = matriz[0,0] + 5
matriz[1,4] = matriz[2,3] * (matriz[2,2]-13)
```

PYTHON

```
import numpy as np

matriz = np.full((3,5), fill_value=0, dtype=float)

matriz[0,0] = 19;
matriz[2,2] = 15;
matriz[2,3] = matriz[0,0] + 5;
matriz[1,4] = matriz[2,3] * (matriz[2,2]-13);
```

RECORRIDO DE MATRICES



ARREGLOS BIDIMENSIONALES



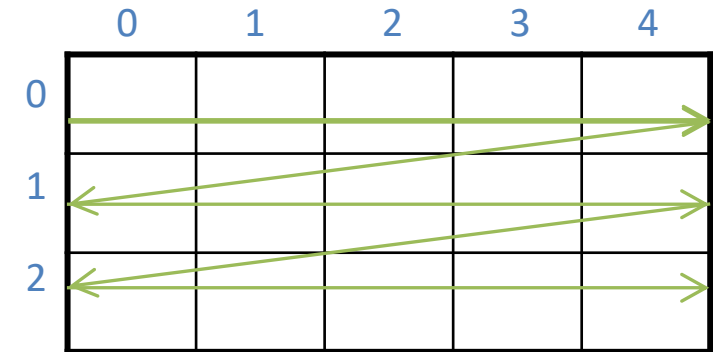
UNIVERSIDAD
DE ANTIOQUIA

- ✈ Existen diferentes formas de recorrer una matriz. Por ejemplo, el siguiente algoritmo llena una matriz recorriendo esta por filas, es decir, primero se llena la fila 0, luego se llena la fila 1, y así sucesivamente:

PSEUDOCÓDIGO

```
publico vacio llenar_por_filas()
    Entero matriz(3,5), FIL=3, COL=5

    Para i=0 hasta FIL haga
        Para j=0 hasta COL haga
            Escribir("Ingrese el valor de celda (",i," ",j,"):")
            Leer(matriz[i,j])
        Fin_Para
    Fin_Para
Fin_Metodo
```



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Existen diferentes formas de recorrer una matriz. Por ejemplo el siguiente algoritmo llena una matriz recorriendo esta por filas, es decir, primero se llena la fila 0, luego se llena la fila 1, y así sucesivamente:

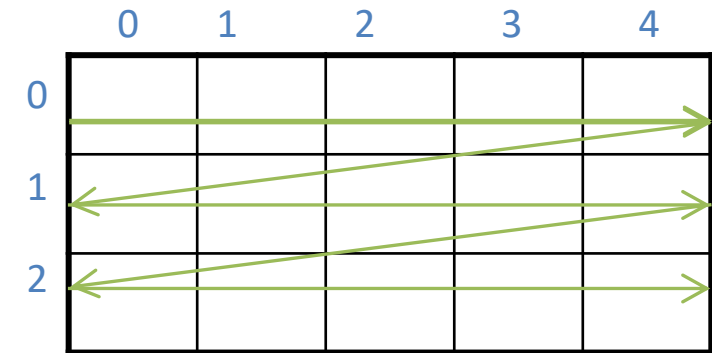
Python

```
import numpy as np

def llenar_por_filas():

    matriz = np.full((3,5), fill_value=0, dtype=int)
    FIL = len(matriz)
    COL = len(matriz[0])

    for i in range(FIL):
        for j in range(COL):
            matriz[i, j] = int(input(f"Ingresa el valor de celda ({i},{j}): "))
```



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✨ Para recorrer una matriz por columnas se cambia el orden de los ciclos, pero se mantienen los índices de la matriz. Recuerde la primera posición es la de la fila y la segunda posición es la de la columna. :

PSEUDOCÓDIGO

```
publico vacio llenar_por_columnas()  
    Entero matriz(3, 5), FIL=3, COL=5
```

```
    Para j=0 hasta COL haga  
        Para i=0 hasta FIL haga
```

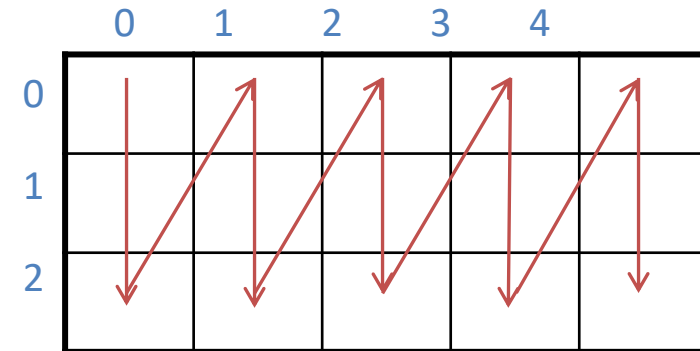
```
            Escribir("Ingrese el valor de celda (",i," ",j,"): ")
```

```
            Leer(matriz[i,j])
```

```
        Fin_Para
```

```
    Fin_Para
```

```
Fin_Metodo
```



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para recorrer una matriz por columnas se cambia el orden de los ciclos, pero se mantienen los índices de la matriz. Recuerde la primera posición es la de la fila y la segunda posición es la de la columna. :

Python

```
import numpy as np
```

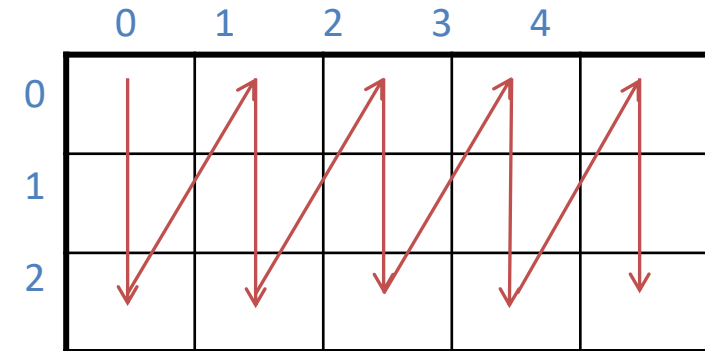
```
def llenar_por_columnas():
```

```
    matriz = np.full((3,5), fill_value=0, dtype=int)
    FIL = len(matriz)
    COL = len(matriz[0])
```

```
    for j in range(COL):
```

```
        for i in range(FIL):
```

```
            matriz[i, j] = int(input(f"Ingresa el valor de celda ({i},{j}): "))
```



- ✈ Para recorrer la diagonal de una **matriz cuadrada** solo se requiere un ciclo. Por ejemplo, asumiendo que se tiene una matriz llena de $N \times N$, el siguiente algoritmo suma todos los elementos de la diagonal:

PSEUDOCÓDIGO

```
publico vacio sumar_elementos_diagonal()
```

```
Entero N=5, matriz(N,N), suma = 0
```

```
Para i=0 hasta N haga
```

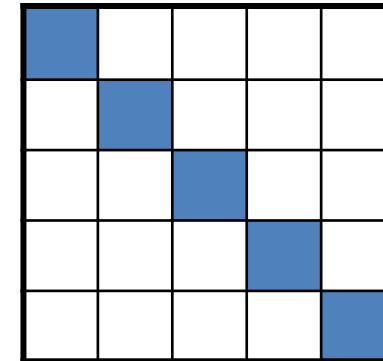
```
    suma = suma + matriz[i,i]
```

```
Fin_Para
```

```
Fin_Metodo
```



La fila y la columna tienen el mismo
valor



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para recorrer la diagonal de una **matriz cuadrada** solo se requiere un ciclo. Por ejemplo, asumiendo que se tiene una matriz llena de $N \times N$, el siguiente algoritmo suma todos los elementos de la diagonal:

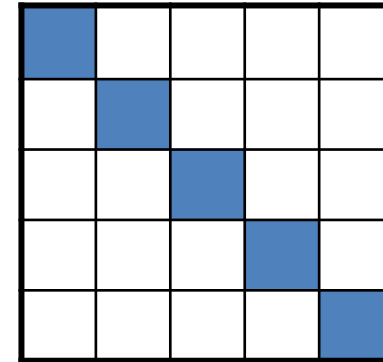
PYTHON

```
import numpy as np

def sumar_elementos_diagonal()

    N = 5
    matriz = np.random.random((N,N))
    suma = 0

    for i in range(N):
        suma = suma + matriz[i,i]
```



- ✈ Para recorrer la diagonal inversa de una **matriz cuadrada** también se requiere de un ciclo. Por ejemplo, asumiendo que se tiene una matriz llena de NxN, el siguiente algoritmo suma todos los elementos de la **diagonal inversa**:

PSEUDOCÓDIGO

```
publico vacío sumar_elementos_diagonal_sec()
```

```
Entero N=5, matriz(N,N), suma = 0
```

```
Para i=0 hasta N haga
```

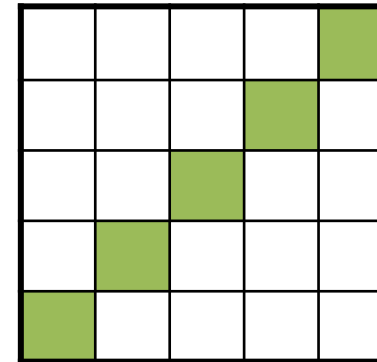
```
    suma = suma + matriz[i, N-1-i]
```

```
Fin_Para
```

```
Fin_Metodo
```



La columna depende de la fila y del
tamaño de la matriz



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para recorrer la diagonal inversa de una **matriz cuadrada** también se requiere de un ciclo. Por ejemplo, asumiendo que se tiene una matriz llena de NxN, el siguiente algoritmo suma todos los elementos de la **diagonal inversa**:

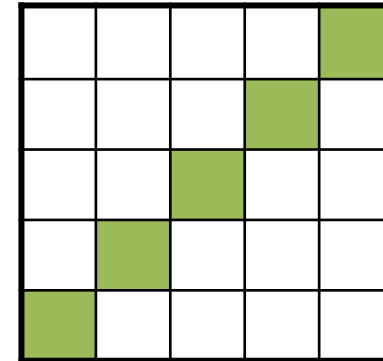
PYTHON

```
import numpy as np

def sumar_elementos_diagonal_sec()

    N = 5
    matriz = np.random.random((N,N))
    suma = 0

    for i in range(N):
        suma = suma + matriz[i,N-1-i]
```



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para recorrer la triangular superior de una **matriz cuadrada** se requiere de dos ciclos ya que se debe hacer un barrido tanto por filas como por columnas. Por ejemplo, asumiendo que se tiene una matriz llena de $N \times N$, el siguiente algoritmo suma todos los elementos de la **triangular superior**:

PSEUDOCÓDIGO

```
publico vacio sumar_triangular_superior()
```

```
Entero N=5, matriz(N,N), suma = 0
```

```
Para i=0 hasta N haga
```

```
Para j=i hasta N haga
```

```
    suma = suma + matriz[i,j]
```

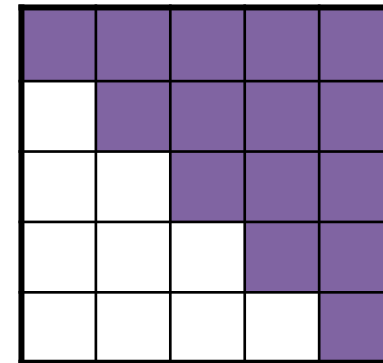
```
Fin_Para
```

```
Fin_Para
```

```
Fin_Metodo
```



Note que la columna inicia en el número de la fila en la que se hace la suma



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

- ✈ Para recorrer la triangular superior de una **matriz cuadrada** se requiere de dos ciclos ya que se debe hacer un barrido tanto por filas como por columnas. Por ejemplo, asumiendo que se tiene una matriz llena de NxN, el siguiente algoritmo suma todos los elementos de la **triangular superior**:

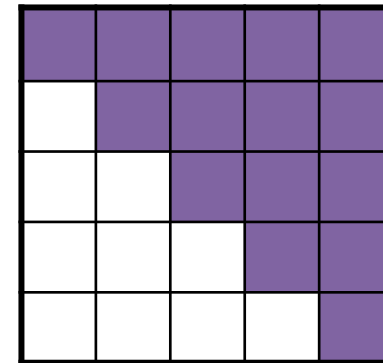
PYTHON

```
import numpy as np

def sumar_triangular_superior()

    N = 5
    matriz = np.random.random((N,N))
    suma = 0

    for i in range(N):
        for j in range(i, N):
            suma = suma + matriz[i,j]
```





EJERCICIO: Haga un algoritmo que pida el nombre y las 4 notas parciales de 50 estudiantes del ITM. El algoritmo debe almacenar el nombre en un arreglo y las notas en una matriz. Adicionalmente, se debe crear un nuevo arreglo que almacene la nota final del estudiante promediando las 4 notas parciales.

El algoritmo debe mostrar el nombre y la nota de los estudiantes que obtuvieron la nota más alta y más baja.



PSEUDOCÓDIGO

Clase NotasEstudiantes

publico vacio principal()

Entero: N=50, numNotas=4, i, j

Real: notasParciales(N,numNotas), notasFinales(N), sumaNotas=0, notaMax, notaMin

Cadena: nombres(N)

Para i=0 **hasta** N **haga**

Escribir("Ingrese el nombre del estudiante # ", i+1, " : ")

Leer(nombres[i])

 sumaNotas = 0;

Para j=0 **hasta** numNotas **haga**

Escribir("Ingrese la nota # ", j+1, " de ", nombres[i])

Leer(notasParciales[i][j])

 sumaNotas = sumaNotas + notasParciales[i][j]

Fin_Para

 notasFinales[i] = sumaNotas/numNotas

Fin_Para

notaMax = notasFinales[0]

notaMin = notasFinales[0]

Para i=0 **hasta** N **haga**

Si (notasFinales[i] > notaMax) **entonces**

 notaMax = notasFinales[i]

Fin_Si

Si (notasFinales[i] < notaMin) **entonces**

 notaMin = notasFinales[i]

Fin_Si

Fin_Para

```
Escribir("La nota más alta del curso fue ", notaMax, " y la obtuvieron los siguientes estudiantes:");  
Para i=0 hasta N haga  
    Si (notasFinales[i] == notaMax) entonces  
        Escribir(nombres[i])  
    Fin_Si  
Fin_Para  
  
Escribir("La nota más baja del curso fue ", notaMin, " y la obtuvieron los siguientes estudiantes:");  
Para i=0 hasta N haga  
    Si (notasFinales[i] == notaMin) entonces  
        Escribir(nombres[i])  
    Fin_Si  
Fin_Para  
Fin_Metodo  
Fin_Clase
```

ACUMULADORES Y PROMEDIOS SOBRE FILAS O COLUMNAS





EJERCICIO: En un hato hay 5 vacas. Haga un algoritmo que permita ingresar la producción de leche de las 5 vacas durante los 7 días de una semana. El algoritmo debe mostrar la producción total de leche de cada vaca y las vacas que tuvieron una producción semanal por encima del promedio.

Tenga presente que los nombres de las vacas son: lola, manchas, colorada, pepita, clara.



ARREGLOS BIDIMENSIONALES



- 🦋 Los **ACUMULADORES Y PROMEDIOS** que se hacen sobre las filas o columnas implican hacer un recorrido de la matriz por filas o columnas, según sea el caso. Se debe tener en cuenta que cuando se desea conocer la suma de los elementos de una fila o columna, o sacar su promedio, se debe inicializar la suma al inicio de cada fila o columna:

Suma = 0; →	2.8	3.0	4.7	5.0	2.5
Suma = 0; →	3.9	4.1	3.2	4.0	3.0
Suma = 0; →	4.7	2.3	5.0	3.6	4.9

ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

👤 **EJEMPLO:** En un hato hay 5 vacas. Haga un algoritmo que permita ingresar la producción de leche de las 5 vacas durante 7 días de una semana. El algoritmo debe mostrar la producción total de leche de cada vaca (**acumulador de filas**).

Los días de la semana se representan
por las columnas, por lo tanto, hay **7 Columnas**

Las vacas se representan por
las filas, por lo tanto, hay **5 Filas**

Matriz de 5x7

ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

Clase ProducciónDeLechePorVaca

publico **vacio** procesar()

Entero i, j, suma = 0, fil=5, col=7, leche(fil,col)

#Ciclo para llenar la matriz

Para i=0 **hasta** fil **haga**

Para j=0 **hasta** col **haga**

 Escribir("Ingrese la producción de la vaca ", i+1," en el día ",j+1," : ")

 Leer(leche[i][j])

Fin_Para

Fin_Para

#Ciclo para calcular la producción por vaca. El recorrido se hace por FILAS

Para i=0 **hasta** fil **haga**

 suma = 0

Al iniciar cada fila la suma
se hace 0

Para j=0 **hasta** col **haga**

 suma = suma + leche[i][j]

Fin_Para

Al terminar la fila se
muestra el acumulador

 Escribir("La vaca ", i+1," produjo ",suma, " litros de leche en la semana ")

Fin_Para

Fin_Metodo

Fin_Clase

ARREGLOS BIDIMENSIONALES

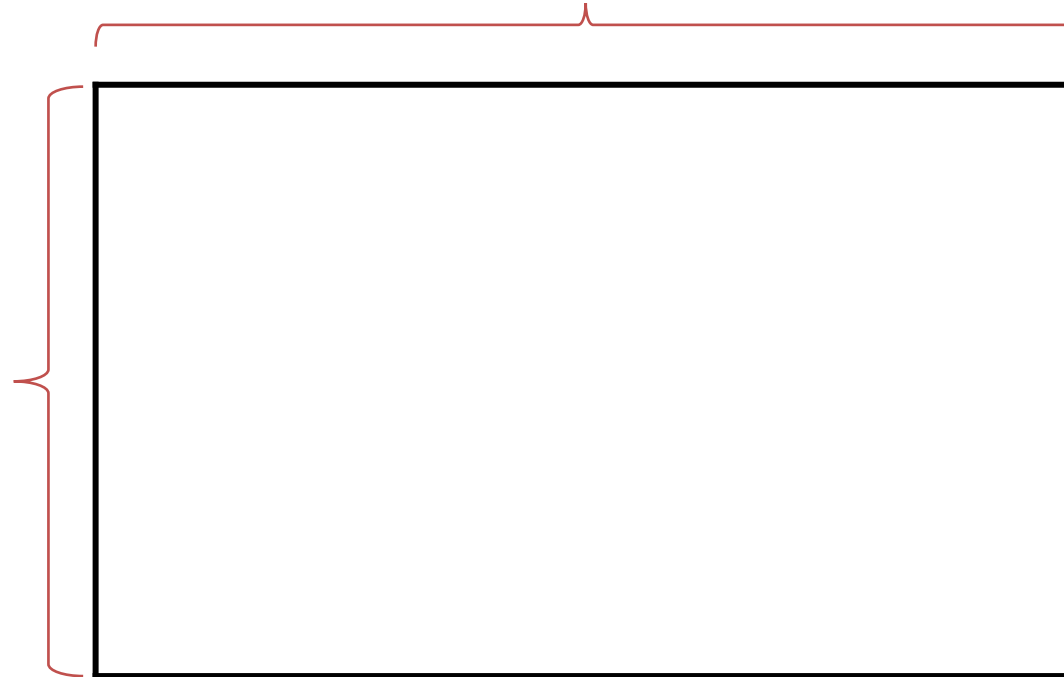


UNIVERSIDAD
DE ANTIOQUIA

- ✂ **EJEMPLO:** En una empresa transportadora hay 15 taxis. Haga un algoritmo que permita ingresar la distancia recorrida de cada taxi durante los 30 días de un mes. El algoritmo debe mostrar la distancia promedio recorrida por día entre todos los taxis (**promedio por cada columna**).

Los días del mes se representan
por las columnas, por lo tanto, hay **30 Columnas**

Los taxis se representan por
las filas, por lo tanto, hay **15 Filas**



Matriz de 15x30

ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

```
clase DistanciaPromedioEntreLosTaxisPorDía
```

```
publico vacio procesar()
```

```
Entero i, j, suma = 0, promedio, fil=15, col=30, dist(fil,col)
```

```
Real promedio
```

```
#Ciclo para llenar la matriz
```

```
Para i=0 hasta fil haga
```

```
    Para j=0 hasta col haga
```

```
        Escribir("Ingrese la distancia recorrida del taxi ", i+1," en el día ",j+1," : ")
```

```
        Leer(dist[i][j])
```

```
    Fin_Para
```

```
Fin_Para
```

```
#Ciclo para calcular el promedio por día. El recorrido se hace por COLUMNAS.
```

```
Para j=0 hasta col haga
```

```
    suma = 0
```

→ Al iniciar cada columna la
suma se hace 0

```
    Para i=0 hasta fil haga
```

```
        suma = suma + dist[i][j]
```

```
    Fin_Para
```

```
    promedio = suma/fil
```

```
    Escribir("En el día ", j+1," la distancia promedio recorrida es de ",promedio," KM, por taxi")
```

↓ Al terminar la columna se calcula y
muestra el promedio

```
Fin_Para
```

```
Fin_Metodo
```

```
Fin_Clase
```

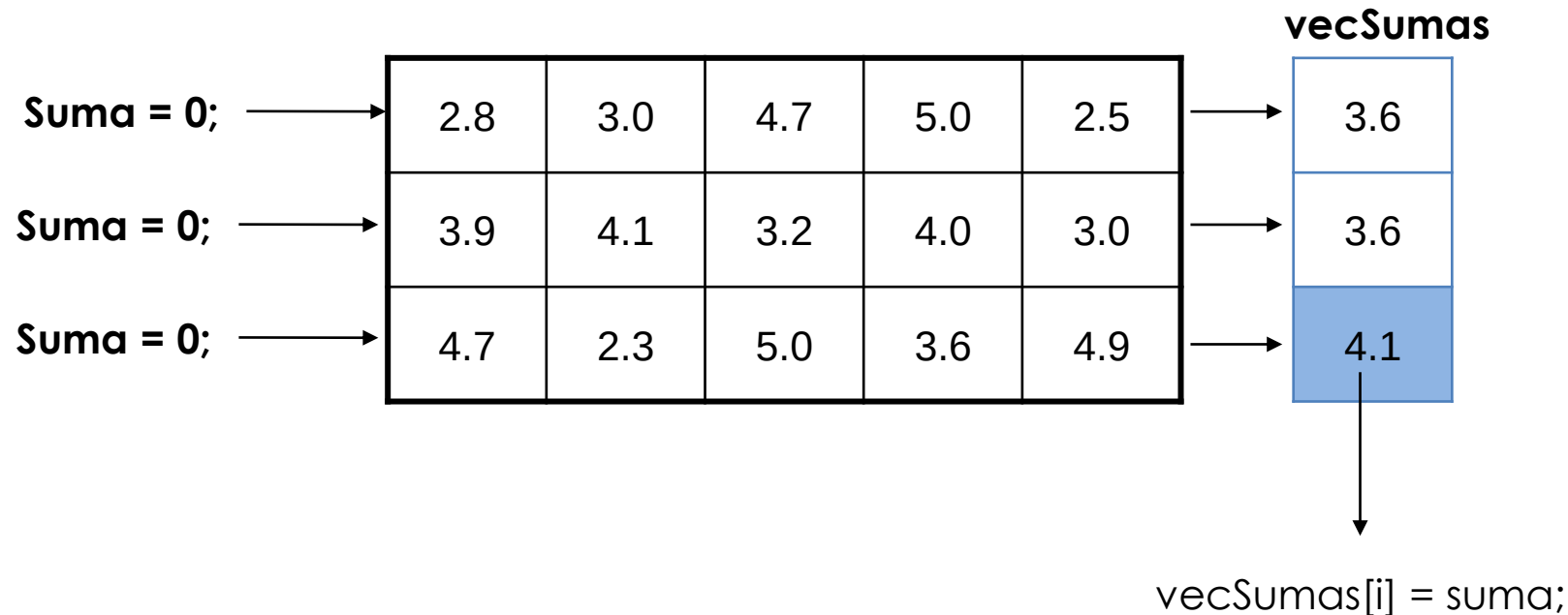
MÁXIMOS O MÍNIMOS DE LOS PROMEDIOS O
ACUMULADORES SOBRE FILAS O COLUMNAS



ARREGLOS BIDIMENSIONALES



- ✂ Para encontrar el **MÁXIMO O EL MÍNIMO** de los **ACUMULADORES Y PROMEDIOS** que se hacen sobre las filas o columnas , estos deben ser almacenados en un vector para luego encontrar el máximo o mínimo de ese vector. Por ejemplo, en el siguiente ejemplo se desea encontrar el máximo entre la suma de cada fila. Por tanto se debe crear un vector que tenga tantas celdas como filas haya en la matriz:

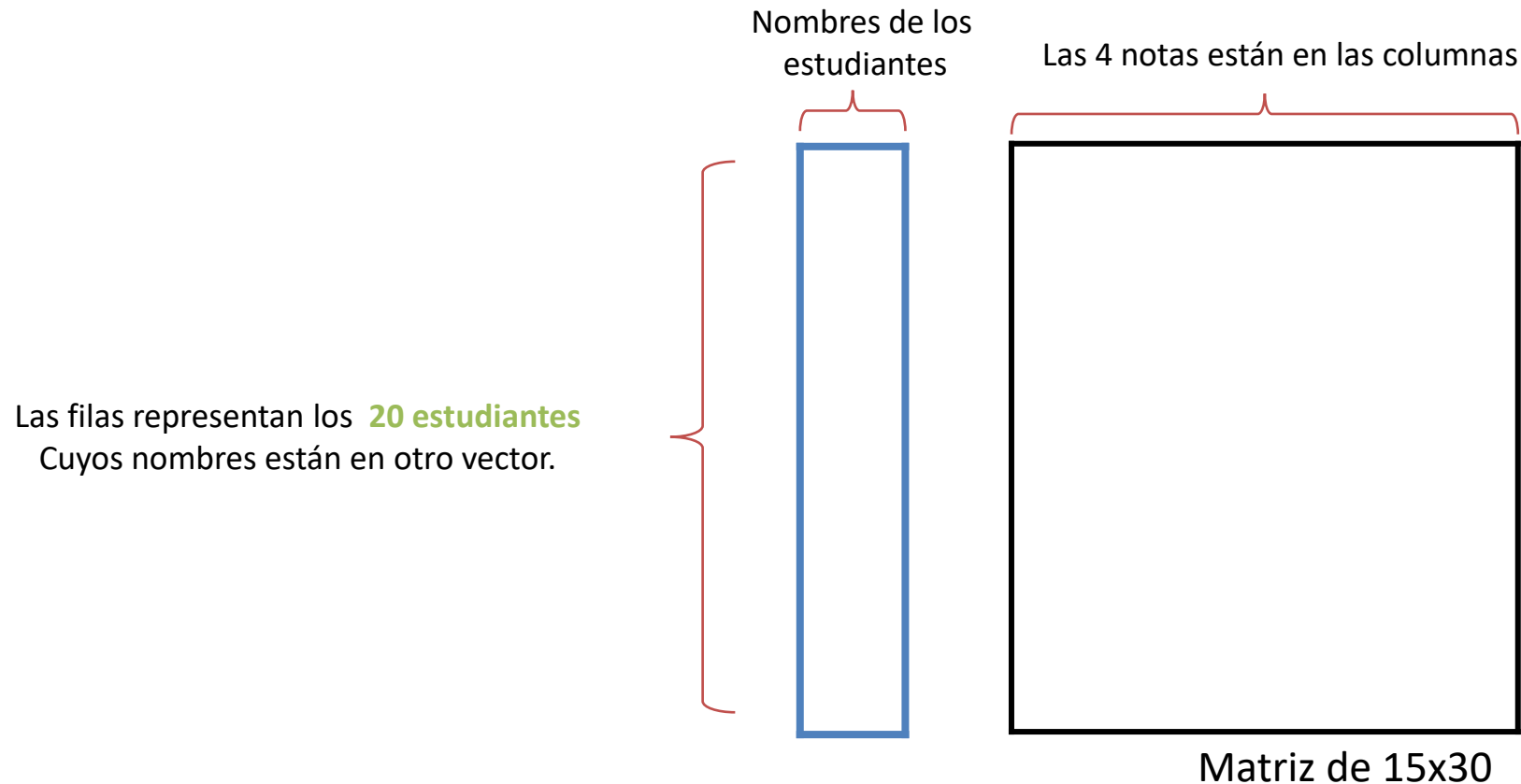


ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

🌟 **EJEMPLO:** En una matriz se almacenan las 4 notas de los 20 estudiantes del curso de fundamentos de programación. Si los nombres de los estudiantes están almacenados en un arreglo llamado `nombresEst` y la matriz es de 20×4 , ¿Cuál es el estudiante con la nota promedio más alta del curso? ([máximo de promedios por filas](#)).



ARREGLOS BIDIMENSIONALES



UNIVERSIDAD
DE ANTIOQUIA

Clase EstudianteConNotaMasAlta

publico **vacio** procesar()

Entero i, j, suma = 0, fil=20, col=4, notas(fil,col), posMax

Real notasPromedio(fil), max

Cadena nombresEst = {...}

#Ciclo para llenar la matriz

Para i=0 **hasta** fil **haga**

Para j=0 **hasta** col **haga**

 Escribir("Ingrese la nota ", j+1, " del estudiante ",
 nombresEst[i], ": ")

 Leer(notas[i][j])

Fin_Para

Fin_Para

#Calcula el promedio de cada estudiante (por FILAS).

Para i=0 **hasta** fil **haga**

 suma = 0

Para j=0 **hasta** col **haga**

 suma = suma + notas[i][j]

Fin_Para

 notasPromedio[i] = suma/col

Fin_Para

#Ciclo para calcular el mayor del arreglo con las notas
promedio

max = notasPromedio[0]

posMax = 0

Para i=0 **hasta** fil **haga**

Si (notasPromedio[i] > max) **entonces**

 max = notasPromedio[i]

 posMax = i

Fin_Si

Fin_Para

#Muestra el estudiante con la nota más alta

Escribir("El estudiante con la nota promedio más alta es:",
nombresEst[posMax], " y su nota fue: ", max)

Fin_Metodo

Fin_Clase

UN PAR DE EJERCICIOS DE REPASO





EJERCICIO: La Secretaria de Medio Ambiente del municipio tiene instalado un sensor de temperatura en el río Medellín, el cual toma una medición por hora de la temperatura que tiene el río. Usted debe realizar un algoritmo que permita almacenar las temperaturas registradas durante los 31 días del mes de mayo y con base en estas se debe mostrar el día en el que el río Medellín estuvo más frío y el día en el que estuvo más caliente, con sus temperaturas respectivas.

El algoritmo también debe mostrar el listado de las temperaturas promedio que se registraron por día ordenadas de menor a mayor.





EJERCICIO: En la UdeA se ha decidido construir una pequeña sala de cine que consta de 12 filas y de 10 asientos cada una. Las filas de la sala de cine están identificadas por las letras desde la A hasta la L y las sillas están numeradas del 1 al 10. Como apenas está iniciando, la sala de cine sólo transmite una función al día, y las boletas solo se vende el mismo día de la función.

Usted ha sido contratado para desarrollar una sencilla aplicación que tenga el siguiente menú y que permita:

1. Ver el mapa de la sala con el estado de cada silla (X llena, O vacía).
2. Vender un número n de boletas consecutivas a una persona, para lo cual la aplicación debe solicitar la fila y la silla de la primera boleto a comprar.
3. Mostrar el resumen del estado de la sala: número sillas vacías y ocupadas de cada fila.
4. Terminar el programa, para lo cual la aplicación deberá indicar: el total de dinero recaudado, asumiendo que cada boleto cuesta \$5000, el número total de sillas libres, el número promedio de sillas ocupadas respecto a las filas y el listado de las filas que quedaron completamente vacías.





**UNIVERSIDAD
DE ANTIOQUIA**

LÓGICA Y REPRESENTACIÓN I

PROGRAMA DE INGENIERÍA DE SISTEMAS

Material desarrollado por **Carlos Andrés Mera Banguero**

<https://github.com/carlosmera20/>